Department of Computer Science — Master Lesson Plan Repository AY 2025-2026

Semesters V & VI | DBMS, Operating Systems, Machine Learning & System Design LLD/HLD

1. Departmental Lesson Plan Governance & Submission Requirements

This Lesson Plan Repository is an official instructional governance document maintained by the Department of Computer Science & Engineering at Northgate Institute. In accordance with NBA Tier-1 accreditation criteria (SAR Section 2.1.1) and AICTE Approval Process Handbook 2024-27, every course instructor must prepare, submit, and execute a structured lesson-by-lesson plan through DCMP before the end of Orientation Week. The plan must include: (a) Lesson-wise objectives mapped to Bloom's Taxonomy level and specific Course Outcomes (COs); (b) teaching methodology (lecture, tutorial, flipped classroom, industry case study, peer review); (c) prescribed and reference texts; (d) lab exercise list with expected outcomes; and (e) approximate mark allocation per unit. Deviations from the approved plan exceeding 20% of contact hours require Head of Department approval.

2. CS301: Database Management Systems — 45-Lecture Blueprint

Instructor: Dr. Elena Marsh | **Credits:** 4 (3L-1T-2P) | **Semesters:** V

Course Outcomes: CO1: Analyze real-world data requirements and design normalized ER schemas (Bloom L4). CO2: Construct and optimize SQL queries for complex retrieval and aggregation tasks (L3). CO3: Apply normalization theory to decompose anomalous relations into BCNF/3NF while preserving dependencies (L3). CO4: Evaluate concurrency control protocols for correctness and performance trade-offs (L5). CO5: Design indexed access paths for performance-optimized query plans (L5).

- **Unit I — Data Models & ER Design (L1–L9):** L1: File systems limitations (redundancy, isolation, integrity, atomicity failure). L2: Three-schema architecture (external/conceptual/internal), data independence (logical, physical). L3: Data model categories (relational, hierarchical, network, object-relational). L4–L5: ER modeling — entities, attributes (simple, composite, multivalued, derived), relationship sets, cardinality constraints (1:1, 1:N, M:N), participation constraints (total, partial), weak entity sets. L6–L7: Extended ER — specialization (disjoint, overlapping), generalization, aggregation, converting EER to relational tables. L8–L9: 7-step ER-to-Relational Mapping algorithm with full worked example (university schema).

- **Unit II — Relational Model & SQL (L10–L18):** L10–L12: Relational Algebra — selection σ, projection π, rename ρ, union ∪, set difference –, Cartesian product ×, natural join ■, theta join, outer joins (left, right, full), division ÷. L13–L14: SQL DDL (CREATE TABLE, ALTER, DROP, CHECK, UNIQUE, FK with CASCADE), DML (SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY, LIMIT). L15–L17: Advanced SQL — correlated subqueries, EXISTS/NOT EXISTS, ALL/ANY, set operations (UNION, INTERSECT, EXCEPT), CTEs (WITH clause), views (updatable conditions), triggers (BEFORE/AFTER, row-level). L18: Window functions (RANK, DENSE_RANK, ROW_NUMBER, NTILE, LAG, LEAD, FIRST_VALUE, LAST_VALUE, OVER clause with PARTITION BY and ORDER BY).

- **Unit III — Normalization Theory (L19–L26):** L19–L21: Functional dependencies, attribute closure algorithm, Armstrong's axioms (proof of soundness and completeness), canonical cover computation (extraneous attribute removal, FD set minimization). L22–L24: Normal forms — 1NF (atomic domains), 2NF (no partial dependencies on CK), 3NF (no transitive dependencies on non-prime attributes), BCNF (every determinant is a superkey), worked examples with relation decomposition. L25–L26: Multi-valued dependencies (MVDs), 4NF, lossless-join property (Abelian group test), dependency preservation property, Bernstein's 3NF synthesis algorithm (detailed worked example).

- **Unit IV — Transaction Processing & Concurrency (L27–L35):** L27–L28: Transaction concept, ACID properties (detailed), transaction states (active, partially committed, committed, failed, aborted), schedules. L29–L30: Serial and non-serial schedules, conflict operations, conflict serializability, precedence graph construction and cycle detection. Recoverability, cascadeless and strict schedules. L31–L33: Locking — shared/exclusive locks, lock compatibility matrix, 2PL (basic, strict, rigorous), lock conversion, MVCC (snapshot isolation, read-your-writes), timestamp ordering protocol. L34–L35: Deadlock handling — prevention (wound-wait, wait-die), detection (wait-for graph, rollback strategy), ARIES logging and recovery (WAL, checkpoint, analysis, redo, undo passes), CLRs.

- **Unit V — Storage, Indexing & Optimization (L36–L45):** L36–L37: Storage organization (heap file, sequential file, clustering), buffer pool management (replacement policies: LRU, Clock, MRU), RAID levels (0, 1, 4, 5, 6). L38–L40: Dense vs sparse indexes, clustered vs unclustered, B+ tree structure and properties, insertion (split propagation), deletion (merge and redistribution), bulk loading. L41–L45: Heuristic query optimization — relational algebra equivalence rules, selection/projection push-down, join ordering. Cost-based optimization fundamentals: statistics (histogram, NDV), selectivity estimation, join size estimation, nested loop join, sort-merge join, hash join algorithms and cost models.

3. CS340: Operating Systems Architecture — 45-Lecture Blueprint

Instructor: Dr. Ravi Subramaniam | **Credits:** 4 (3L-1T-2P) | **Semester:** V

• **Unit I — Process Management & Scheduling (L1–L9):** L1: OS roles and structure (monolithic, microkernel, hybrid, exokernel). L2: Process abstraction, PCB fields, process lifecycle state machine. L3: System calls, context switch overhead analysis. L4: Threading — user-mode vs kernel-mode, M:N hybrid model, POSIX pthreads API. L5–L7: CPU scheduling algorithms — FCFS (convoy effect), SJF/SRTF (preemptive), Round Robin (quantum trade-offs), Priority (starvation and aging), MLFQ design and rules (Ousterhout's 3 rules). L8: Real-time scheduling (EDF, RMS, rate monotonic analysis). L9: Multiprocessor scheduling (load balancing, affinity, NUMA awareness).

• **Unit II — Synchronization & Deadlock (L10–L18):** L10: Race conditions, critical section requirements (mutual exclusion, progress, bounded waiting). L11: Peterson's algorithm (proof of correctness), hardware memory barriers, TestAndSet, CompareAndSwap. L12–L13: Semaphores (binary/counting, P/V operations), mutex locks, monitors (Mesa vs Hoare semantics), condition variables. L14–L15: Classical sync problems — Producer-Consumer (3 semaphore solution), Readers-Writers (3 variants with starvation analysis), Dining Philosophers (Chandy-Misra napkin solution). L16–L17: Deadlock — 4 Coffman conditions, Resource Allocation Graph (single vs multi-instance), Banker's Algorithm (safety check, resource request), detection and recovery (rollback strategies). L18: Lock-free and wait-free data structures (CAS-based stack, Michael-Scott queue), transactional memory.

• **Unit III — Memory Management & Virtual Memory (L19–L28):** L19–L20: Logical vs physical address, static vs dynamic binding, swapping, contiguous allocation (first fit, best fit, worst fit), external vs internal fragmentation. L21–L22: Paging — page table structure, TLB operation, effective access time, multi-level page tables (2-level, 4-level x86-64), inverted page tables. L23–L24: Segmentation, segment descriptor table, segmentation+paging (x86). L25–L26: Virtual memory — demand paging, page fault handling sequence, copy-on-write, thrashing analysis. L27–L28: Page replacement algorithms — FIFO (Belady's anomaly), Optimal (OPT/MIN), LRU (stack algorithm, working set model), Clock/Second-Chance approximation, NRU, Working Set Window. Memory-mapped files, prepaging, page size trade-offs.

• **Unit IV — File Systems & I/O Subsystems (L29–L36):** L29–L30: File abstraction, directory structure (flat, tree, DAG, symlinks), VFS layer, inode structure (direct, indirect, double-indirect pointers), hard links vs symbolic links. L31: File allocation methods (contiguous, linked, indexed, extent-based), free-space management (bitmap, free list, grouping). L32: File system recovery — fsck algorithm, journaling (write-ahead log, ordered journal, full data journal), ext4/NTFS/ZFS internals. L33–L34: I/O subsystem — device drivers, interrupt handling, DMA controller, I/O scheduling for SSDs vs HDDs. L35–L36: Disk scheduling — SSTF, SCAN, C-SCAN, LOOK, C-LOOK; SSD characteristics (wear leveling, FTL, garbage collection, TRIM).

• **Unit V — Security, Protection & Virtualization (L37–L45):** L37–L38: Protection domain, access matrix (ACL, capability list), role-based access control (RBAC), mandatory access control (MAC — Bell-LaPadula, Biba). L39: Program threats (buffer overflow, stack smashing, ROP chains, ASLR mitigation). L40: OS security hardening (SELinux, AppArmor, seccomp-bpf). L41–L42: Virtualization — Type-1 vs Type-2 hypervisors, hardware-assisted virtualization (Intel VT-x, AMD-V, IOMMU), memory ballooning, VM live migration. L43–L44: Container internals — Linux namespaces (pid, net, mnt, uts, ipc, user), cgroups v2 resource limits, overlay filesystems, Docker layered image architecture, Kubernetes pod lifecycle. L45: Cloud OS design, serverless runtime (AWS Lambda firecracker microVMs), unikernel architectures.

4. CS420: Applied Machine Learning & AI — 45-Lecture Blueprint

Instructor: Dr. Ananya Krishnan | **Credits:** 4 (3L-2P) | **Semester:** VI

• **Unit I — Supervised Learning Foundations (L1–L9):** L1–L2: ML paradigm (supervised, unsupervised, self-supervised, RL), PAC learning theory, hypothesis spaces, empirical risk minimization. L3–L4: Linear Regression (OLS derivation, normal equations, closed-form solution, gradient descent update rule, stochastic and mini-batch GD). L5: Regularization (Ridge L2 — Tikhonov, Lasso L1 — sparsity, ElasticNet), bias-variance decomposition proof. L6–L7: Model evaluation — k-fold cross-validation, stratified CV, learning curves, hyperparameter tuning (grid search, random search, Bayesian optimization with Gaussian Processes). L8–L9: Logistic Regression (sigmoid derivation, MLE, softmax multi-class, decision boundaries).

• **Unit II — Classification & Ensemble Methods (L10–L18):** L10–L11: Decision Trees (CART, ID3, C4.5, information gain, entropy, Gini impurity, pruning strategies, minimum description length). L12–L13: Ensemble methods — bagging (variance reduction), Random Forests (feature sub-sampling, out-of-bag error, feature importance). L14–L15: Gradient Boosting — functional gradient descent, AdaBoost, XGBoost (tree structure optimization, L1/L2 regularization, column sub-sampling, weighted quantile sketch), LightGBM (GOSS, EFB, leaf-wise growth), CatBoost (ordered boosting). L16–L17: Support Vector Machines — maximum margin classifier, soft margin (C parameter), kernel trick (Mercer's theorem), RBF kernel, polynomial, sigmoid, dual formulation and KKT conditions. L18: SVM multi-class (OvR, OvO), calibration (Platt scaling, isotonic regression).

• **Unit III — Deep Neural Networks (L19–L27):** L19: Perceptron, MLP motivation, universal approximation theorem. L20: Activation functions — sigmoid, tanh, ReLU (dying ReLU problem), Leaky ReLU, ELU, GELU, Swish, PReLU. L21: Backpropagation — computational graph, chain rule derivation, vanishing/exploding gradients. L22: Weight initialization (Xavier/Glorot for sigmoid/tanh, He/Kaiming for ReLU), batch normalization (internal covariate shift, running mean/variance, γ/β parameters), layer normalization, dropout (Bernoulli mask, inverted dropout). L23: Optimizers — SGD with momentum, RMSProp, Adam (bias correction), AdamW (decoupled weight decay), learning rate schedules (cosine annealing, warm restart, one-cycle). L24–L25: CNNs — convolution operation (cross-correlation, filter banks, feature maps), pooling, receptive field computation, modern architectures (AlexNet, VGG, ResNet — residual connections and vanishing gradient solution, EfficientNet — compound scaling). L26: Sequence models — RNNs (BPTT, vanishing gradient), LSTM (forget, input, output gates, cell state), GRU (simplified gating). L27: Attention mechanism (scaled dot-product, multi-head attention), Transformer encoder-decoder, positional encoding, BERT (masked LM, NSP), GPT (autoregressive language modeling), parameter-efficient fine-tuning (LoRA, prefix tuning).

• **Unit IV — Unsupervised Learning & Generative Models (L28–L36):** L28: Clustering objectives, K-Means algorithm (Lloyd's), K-Means++ initialization, Elbow method, Silhouette score. L29: DBSCAN (ε-neighborhood, MinPts, core/border/noise points, algorithm complexity), HDBSCAN. L30: Hierarchical clustering (agglomerative with dendrograms, Ward/average/complete linkage). L31: Dimensionality reduction — PCA (covariance eigendecomposition, scree plot, variance explained, reconstruction error), kernel PCA, LDA. L32: Non-linear DR — t-SNE (KL divergence minimization, perplexity, crowding problem), UMAP (topological data analysis, manifold learning). L33: Anomaly detection — Isolation Forest, One-Class SVM, Autoencoder reconstruction error. L34–L35: Generative models — Variational Autoencoders (ELBO, reparameterization trick), GANs (min-max game, mode collapse, Wasserstein GAN), Diffusion Models (DDPM, score matching, DDIM). L36: Large-scale clustering and vector databases for RAG applications (Faiss, Qdrant, Weaviate).

• **Unit V — Responsible AI, Evaluation & Production (L37–L45):** L37–L38: Evaluation metrics — confusion matrix, Precision/Recall/F1, Matthews CC, ROC-AUC, PR curve, Cohen's Kappa, calibration (reliability diagram, ECE, MCE). L39: Class imbalance handling — SMOTE, ADASYN, cost-sensitive learning, threshold optimization. L40: Responsible AI — fairness metrics (demographic parity, equalized odds, individual fairness), bias auditing (AIF360), model documentation (model cards). L41: Explainability — SHAP (TreeSHAP, KernelSHAP), LIME, Integrated Gradients, attention visualization, saliency maps. L42–L43: ML Ops — experiment tracking (MLflow), model registry, feature stores, data versioning (DVC), CI/CD for ML (GitHub Actions + Docker). L44–L45: LLMs in production — retrieval-augmented generation (RAG) architecture, vector search (ANN algorithms: HNSW, IVF), embedding models, chunking strategies, re-ranking, evaluation (RAGAs framework).

5. CS460: System Design LLD & HLD — Lesson-by-Lesson Blueprint

Instructor: Dr. Vikram Nair | **Credits:** 4 (3L-2 Design Studio) | **Semester:** VI

• **Lesson 1 — System Design Fundamentals & Scalability:** Vertical scaling (CPU/RAM upgrade limits), horizontal scaling (commodity hardware, stateless service design). Latency vs throughput (Little's Law: L = λW), tail latency percentiles (P50/P95/P99), SLA vs SLO vs SLI. Load balancing — Layer-4 (TCP/IP) vs Layer-7 (HTTP/gRPC), algorithms (round-robin, least-connections, IP hash, consistent hashing, rendezvous hashing). Consistent hashing ring: virtual nodes, load distribution analysis. CAP Theorem: network partition unavoidability proof, CP systems (HBase, Zookeeper, etcd), AP systems (Cassandra, CouchDB, DynamoDB). PACELC extension. BASE properties. High availability patterns: active-passive vs active-active, multi-AZ deployment.

• **Lesson 2 — Low-Level Design: SOLID & OOP:** Encapsulation, abstraction, inheritance (fragile base class problem), polymorphism (subtype and parametric). SOLID principles in depth: SRP (cohesion metric, single axis of change), OCP (extension without modification — strategy pattern implementation), LSP (Liskov substitution formal definition, covariance/contravariance), ISP (fat interface anti-pattern, role interfaces), DIP (dependency injection frameworks — Spring, Guice). Composition over inheritance principle. YAGNI and DRY principles. UML modeling: Class diagram (association, aggregation, composition, dependency, realization, generalization multiplicities), Sequence diagram (lifelines, messages, activation bars, alt/loop/opt fragments), State Machine diagram, Activity diagram.

• **Lesson 3 — Low-Level Design: Design Patterns Practicum:** Creational: Thread-safe Singleton (double-checked locking, Bill Pugh initialization-on-demand), Factory Method (defer instantiation to subclasses), Abstract Factory (product families), Builder (fluent API, Director class), Prototype (deep vs shallow clone). Structural: Adapter (object vs class adapter), Bridge (decouple abstraction from implementation), Composite (tree structures, uniform leaf/branch treatment), Decorator (dynamic responsibility addition, Java I/O streams), Facade (subsystem simplification), Proxy (virtual, remote, protection proxy types). Behavioral: Strategy (algorithm family encapsulation — sorting strategies, payment processors), Observer (Event bus, reactive programming, Pub-Sub with weak references), Command (undo/redo history, macro recording), Template Method (Hollywood principle), Iterator (internal vs external, lazy sequences), State Machine (vending machine, order state transitions), Mediator (chat room, air traffic control), Chain of Responsibility (logging filter chain, HTTP middleware pipeline). Full case study: Enterprise Parking Lot System design — entry/exit

gates, vehicle types (two-wheeler, four-wheeler, heavy vehicle), spot allocation strategy, pricing engine (hourly/flat-rate/subscription), ParkingTicket, PaymentProcessor, SecurityCamera integration.

• **Lesson 4 — High-Level Design: Storage, Sharding & Caching:** SQL vs NoSQL decision framework (ACID requirements, schema flexibility, query patterns, scale). NoSQL taxonomy: document (MongoDB), key-value (Redis, DynamoDB), columnar (Cassandra, HBase), graph (Neo4j). Replication: synchronous (strong consistency, write latency), asynchronous (eventual consistency, replication lag), semi-synchronous; leader-follower vs multi-leader vs leaderless (quorum: W+R > N). Database sharding strategies: range sharding (hot shard problem), hash sharding (modulo vs consistent hash), directory-based sharding (shard map service). Cross-shard joins and distributed transactions overhead. Caching topologies: Cache-Aside (lazy loading), Read-Through (synchronous population), Write-Through (strong consistency), Write-Behind (async flush, durability risk). Eviction policies: LRU (LinkedHashMap implementation), LFU (min-heap + frequency map), CLOCK. Redis data structures (string, hash, list, set, sorted set, stream, geo) and their use cases. CDN edge caching: cache-control headers, TTL, origin shield, invalidation strategies.

• **Lesson 5 — High-Level Design: Distributed Systems & Microservices:** Monolith decomposition patterns: Strangler Fig, Anti-Corruption Layer, Branch by Abstraction. Domain-Driven Design: bounded contexts, context maps, aggregates, domain events. API Gateway: authentication (JWT/OAuth2), rate limiting (Token Bucket, Leaky Bucket, Sliding Window Counter), request aggregation, SSL termination. Circuit Breaker pattern (Closed/Open/Half-Open states, failure threshold, Hystrix/Resilience4j). Service Mesh: sidecar proxy (Envoy), mTLS, observability (Prometheus metrics, Jaeger distributed tracing, ELK logs). Message queues: Apache Kafka architecture (brokers, topics, partitions, consumer groups, ISR, log compaction, exactly-once semantics), RabbitMQ (exchanges, bindings, AMQP). Distributed transactions: Two-Phase Commit (coordinator failure problem), SAGA pattern (orchestration vs choreography, compensating transactions). Distributed consensus: Raft protocol (leader election, log replication, commit rule, safety proof), Paxos comparison. API design: REST constraints (HATEOAS, versioning strategies), gRPC (Protocol Buffers, bidirectional streaming), GraphQL (N+1 problem, DataLoader batching). Idempotency keys for exactly-once client retries.

• **Lesson 6 — End-to-End System Design Case Studies:** (1) Distributed URL Shortener (TinyURL scale: 100M writes/day, 1B reads/day): Base62 encoding, Key Generation Service (pre-generated key pool), Redis cache for hot URLs, Cassandra for URL metadata, CDN for redirect headers, analytics pipeline (Kafka → Flink → ClickHouse). (2) Real-Time Push Notification System (1B subscribers, 10M pushes/minute): WebSocket connection server with sticky load balancing, FCM/APNS integration, notification deduplication (Redis Bloom Filter), priority queues (Kafka), retry with exponential backoff, delivery receipt tracking. (3) Distributed Online Examination Proctoring Platform: Auto-scaling WebRTC streaming servers (Kubernetes HPA), AI-based anomaly detection (eye-tracking, background noise analysis), immutable blockchain audit log for exam answers (Hyperledger Fabric), S3 video recording with lifecycle policies, real-time grading pipeline. (4) Social Photo-Sharing Platform (Instagram scale: 500M DAU): Hybrid feed generation (push fanout for celebrities, pull for normal users, threshold-based switching), CDN image storage with WebP transcoding, Cassandra denormalized follow graph, Elasticsearch for hashtag search.

6. Active Learning Pedagogies, Lab Rotation & Reference Texts

All courses integrate flipped classroom pre-lecture videos (15-minute YouTube playlists), peer code review sessions (pair programming rubric), industry guest lectures (minimum 3 per semester per course from T&P; industry panel), and competitive programming practice (LeetCode company-tagged problems aligned to each unit). Reference Texts: (1) Silberschatz, Korth & Sudarshan — Database System Concepts, 7th Ed.; (2) Silberschatz, Galvin & Gagne — Operating System Concepts, 10th Ed.; (3) Christopher Bishop — Pattern Recognition and Machine Learning; (4) Ian Goodfellow, Bengio & Courville — Deep Learning; (5) Martin Kleppmann — Designing Data-Intensive Applications; (6) Alex Xu — System Design Interview: An Insider's Guide (Vols. I & II); (7) Gamma, Helm, Johnson & Vlissides — Design Patterns: Elements of Reusable OO Software; (8) Michael Nygard — Release It! Design and Deploy Production-Ready Software, 2nd Ed.